There are a couple of different ways to think about a test double. One is the *usage mode* of the double—that is, what you're using it for and what you're expecting it to do. The other thing to consider is how the double is created. We'll call this the double's *origin*.

Here are the usage modes we'll be talking about in this chapter:

Stub
: Returns canned responses, avoiding any meaningful computation or I/O

Mock
: Expects specific messages; will raise an error if it *doesn't* receive them by the end of the example

Null Object
: A benign test double that can stand in for any object; returns itself in response to any message

Spy
: Records the messages it receives, so that you can check them later

We're basing the terms here on the vocabulary that Gerard Meszaros developed in his book, *xUnit Test Patterns [Mes07]*.

In addition to having a usage mode, a test double has an *origin*, indicating what its underlying Ruby class is. Some doubles are based on real Ruby objects, and some are totally fake:

Pure Double
: A double whose behavior comes entirely from the test framework; this is what people normally think of when they talk about mock objects

Partial Double
: An existing Ruby object that takes on some test double behavior; its interface is a mixture of real and fake implementations

Verifying Double
: Totally fake like a pure double, but constrains its interface based on a real object like a partial double; provides a safer test double by verifying that it matches the API it's standing in for

Stubbed Constant
: A Ruby constant—such as a class or module name—which you create, remove, or replace for a single test